

ARGUS: Production-Scale Tracing and Performance Diagnosis for over 10,000-GPU Clusters

Jiasheng Zhou
Tencent

Longbin Zeng
Tencent

Clavis Chen
Tencent

Ruiming Lu
Tencent

Qinwei Yang
Tencent

Leyi Ye
Tencent

Ray Ying
Tencent

Key Zhang
Tencent

Abstract

Large-scale LLM training requires always-on, fine-grained observability for effective performance diagnosis at scale. Coarse resource monitors alone cannot localize root causes, and fine-grained profilers incur prohibitive (5%–30%) overheads and massive trace volumes, making always-on deployment impractical in large production clusters.

We propose ARGUS, a low-overhead, fine-grained, always-on tracing and real-time analysis system for training workloads in 10,000+ GPU-scale production clusters. ARGUS decomposes observation along the training call hierarchy into CPU call stacks, framework semantics, and GPU kernel execution, with always-on collection under a combined overhead of less than 2%. It builds a unified data pipeline and compresses raw kernel events by approximately 3,700 $\times$ from 10 MB to 2.7 KB per rank per step. Its progressive diagnosis framework automatically isolates anomalous windows, straggler ranks, and degraded kernels through iteration-time, phase-level, and kernel-level analysis. Deployed for over six months on a 10,000+ GPU production cluster, ARGUS has supported continuous fail-slow detection and performance optimization. Our case studies further demonstrate its effectiveness across representative anomalies, including compute stragglers, link degradation, pipeline-bubble amplification, FlashAttention JIT stalls, and compute stragglers masked by communication symptoms.

1 Introduction

In recent years, the rapid development of large language models (LLMs) [1–3, 22, 35] has driven a continuous expansion of training infrastructure. Today, mainstream LLMs reach hundreds of billions to trillions of parameters [13]. A single pre-training run typically occupies thousands to tens of thousands of GPUs for weeks or even months [18, 21, 41]. This scale transforms performance issues from occasional incidents into systematic challenges: in a 10,000-GPU cluster, performance degradation in any single component can slow down the overall training progress, and diagnosis grows super-linearly harder with scale [23, 24, 42, 43].

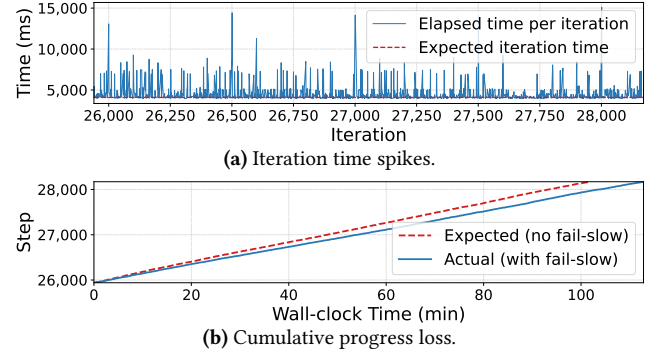


Figure 1. Fail-slow in a 4096-GPU training job.

Performance diagnosis for large-scale training encompasses two complementary aspects: localizing *fail-slow* faults, and identifying *performance optimization* opportunities. Unlike fail-stop failures that halt execution, fail-slow refers to performance degradation in any component—such as GPU hardware, communication fabric, or host-side software—that drags down the entire synchronous training job without triggering explicit errors. As shown in Figure 1, in a 4096-GPU training job, iteration time exhibits numerous spikes exceeding twice the expected duration, wasting approximately 23,758 GPU-hours (7% of total training compute). This problem is stealthy, random, and difficult to reproduce, limiting large-scale training efficiency.

However, existing monitoring and diagnosis approaches are inadequate for effective performance diagnosis. The first class (Greyhound [42], Holmes [43], C4 [11], Minder [9], ByteRobust [41], Mycroft [10]) adopts always-on continuous operation, tracking iteration time, communication operators, or infrastructure metrics at low overhead. They can detect anomalies and localize them to a specific machine or link, but cannot answer which kernel slowed down and why. The second class (MegaScale [21], EROICA [17], FLARE [5]) provides finer-grained traces, but each with several limitations: some remain at phase-level granularity and rely on manual post-hoc analysis; some trigger kernel-level profiling only after detecting anomalies with runtime overhead of 5%–30% or more, and cannot be kept continuously enabled in production training. Thus, no existing system can simultaneously

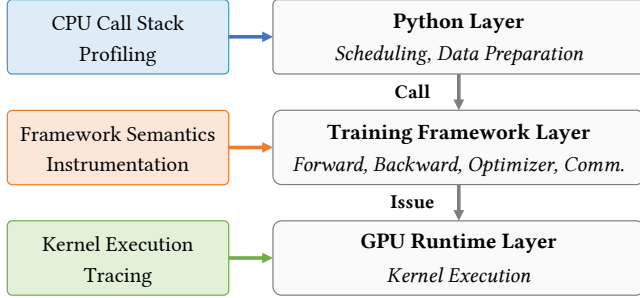


Figure 2. The hierarchical structure of training execution.

achieve fine granularity, always-on operation, and real-time cross-rank analysis at 10,000-GPU scale.

Building such a system faces two core challenges. First, there is an inherent tension between fine-grained observation and low overhead. Comprehensive observation of the full training execution introduces significant runtime overhead. This not only slows training but also creates an observer effect, causing traces to capture perturbed rather than original execution behavior. Second, fine-grained trace data at 10,000-GPU scale is extraordinarily voluminous. In a 10,000-GPU cluster, each GPU generates 10^4 to 10^5 kernel events per minute, producing over 1 GB/min of raw traces cluster-wide. Performing cross-rank online comparison directly on the full raw data is computationally infeasible.

To address these challenges, we propose ARGUS, a low-overhead, fine-grained, always-on tracing and real-time analysis system for large-scale training workloads. Its design is guided by the call hierarchy of modern training systems. As shown in Figure 2, a training iteration spans three layers: the Python layer for scheduling and data preparation, the framework layer (e.g., Megatron [27, 37], ZeRO [34], and FSDP [44]) for phase orchestration, and the GPU runtime layer for kernel execution. Since performance anomalies may arise at any layer and manifest differently, ARGUS decomposes observability into three corresponding signals: CPU call stacks, framework semantics, and GPU kernel traces. This decomposition enables always-on collection with a total overhead below 2%. To support real-time analysis at 10,000-GPU scale, ARGUS builds a unified data pipeline that streams structured metrics to a time-series database for monitoring and alerting, while persisting converted raw traces to object storage for offline analysis. For voluminous kernel traces, ARGUS further introduces an online statistical compression method based on kernel density estimation (KDE) clustering [36], reducing events in each time window into KB-scale summaries with a compression ratio of 10^3 – 10^4 . On top of this pipeline, ARGUS implements a progressive diagnosis framework across different granularities, narrowing manual analysis from tens of thousands of GPUs to a few ranks and time windows. ARGUS has been deployed in a production cluster of over 10,000 GPUs. The main contributions of this paper are as follows:

- We design and implement a low-overhead, fine-grained, always-on tracing system for 10,000-GPU scale clusters, decomposing observation into three independent mechanisms (CPU call stack, framework semantics, and kernel execution) that span from host-side behavior to GPU kernel execution, with a total overhead of less than 2% (§4).
- We build a unified data pipeline that supports real-time transport, tiered storage, and online analysis. For the highest-volume kernel execution traces, we propose an online statistical compression method based on KDE clustering that achieves approximately 3,700× compression (from 10 MB to 2.7 KB per rank per step), enabling online cross-rank anomaly detection at 10,000-GPU scale (§5).
- We design a progressive diagnosis framework with parallel detection levels spanning iteration-time anomaly detection, cross-rank attribution, and kernel-level distribution comparison, narrowing the diagnostic scope from tens of thousands of ranks to single-digit suspects (§6).
- We deploy ARGUS on a production cluster of over 10,000 GPUs for more than six months, and demonstrate its practical effectiveness through five real-world case studies, diagnosing compute stragglers, communication link degradation, pipeline bubble amplification, JIT compilation blocking, and compute stragglers masked by communication symptoms (§7, §8).

2 Motivation and Design Space

2.1 Motivation

The fail-slow problem in large-scale LLM training is not a simple training interruption, but rather the performance degradation of a few ranks, GPUs, links, or host-side components in synchronous training that drags down overall progress [24, 42]. Existing resource-level monitoring can detect the presence of anomalies but cannot explain *why*: it cannot distinguish whether a kernel has become slower, communication is blocked, a GPU idle gap has appeared, data loading is stalled, or Python GC is interfering.

A tracing and diagnosis system for 10,000-GPU scale clusters must simultaneously satisfy multiple interrelated requirements. At the system-property level, GPU time is extremely expensive and any monitoring overhead directly translates into wasted compute, so the system must maintain *low overhead*. At the same time, merely identifying which machine slowed down is insufficient to guide remediation—the system needs multi-level *fine granularity* observability spanning from training semantics to individual kernels. Furthermore, fail-slow events are intermittent and unpredictable, requiring the system to operate *always-on* rather than relying on manual triggering or short-window sampling. At 10,000-GPU scale, every second of performance regression wastes substantial compute, demanding *real-time analysis* capability that delivers diagnostic conclusions within minutes. At the diagnostic-capability level, the system must automatically

Table 1. Capability comparison of existing systems. ✓ indicates full support, △ indicates partial support or with limitations, and ✗ indicates no support. Analysis timeliness: *Cont.*=continuous, *Trig.*=triggered after anomaly detection.

System	System Properties				Diagnostic Capabilities	
	Low overhead	Fine granularity	Always-on	Real-time analysis	Fail-slow localization	Perf. optimization
Greyhound [42]	✓	✗	✓	Trig.	✓	✗
Holmes [43]	✓	✗	✓	Trig.	✓	✗
C4 [11]	✓	✗	✓	Cont.	△	✗
Minder [9]	✓	✗	✓	Cont.	△	✗
ByteRobust [41]	✓	✗	✓	Cont.	△	✗
Mycroft [10]	✓	△	✓	Cont.	△	△
MegaScale [21]	✓	✗	✓	Trig.	△	✓
EROICA [17]	△	✓	✗	Trig.	✓	✓
FLARE [5]	✓	✓	✓	Cont.	△	✓
ARGUS	✓	✓	✓	Cont.	✓	✓

identify stragglers among over 10,000 ranks and progressively narrow the scope for *fail-slow localization*, while also supporting parallel-strategy configuration, communication-computation overlap analysis, and operator efficiency evaluation for *performance optimization* [21, 45].

2.2 Limitations of Existing Systems

As shown in Table 1, existing methods can be broadly categorized into two approaches, neither fully satisfying all of the above requirements. The first class of methods (Greyhound [42], Holmes [43], C4 [11], Minder [9], ByteRobust [41], Mycroft [10], *etc.*) choose always-on continuous operation, tracking iteration time, communication operator duration, communication-layer dependencies, or infrastructure metrics at low overhead. CCL-customized systems such as Mycroft and Aegis [12] further improve runtime diagnosis, but remain primarily bounded by communication-layer observability. They can rapidly detect anomalies and localize them to a specific machine, link, or communication dependency, but cannot answer which arbitrary GPU kernel slowed down and why. The second class of methods (MegaScale [21], EROICA [17], FLARE [5], *etc.*) provide finer-grained traces, but each with different limitations: MegaScale remains at phase-level granularity and relies on manual post-hoc analysis; EROICA triggers kernel-level profiling only after detecting anomalies, potentially missing critical windows of sporadic events; FLARE restricts coverage to a predefined operator set with coverage bounded by explicitly instrumented operators. More importantly, although systems like C4, Minder, and FLARE achieve continuous analysis, their diagnostic granularity remains limited to machine-level, link-level, or predefined operator-level identification. No existing system can perform kernel-level online cross-rank comparison at 10,000-GPU scale: the first class lacks the observability depth to pinpoint which specific kernel degraded, while the second class’s trace data volume makes online cross-rank comparison infeasible. In summary, existing systems either run continuously but lack sufficient diagnostic fidelity, or provide fine-grained traces but cannot be continuously deployed and scaled to 10,000+ GPUs for online cross-rank analysis—no single system satisfies all requirements in §2.1.

2.3 Challenges

To realize a 10,000-GPU scale tracing system that simultaneously achieves fine-grained, always-on, and real-time capabilities, two core challenges must be addressed.

Challenge 1: Observer effect under a strict overhead budget. Accurately localizing fail-slow requires multi-level fine-grained observation of the training execution process. General-purpose profilers and fine-grained GPU kernel profilers can achieve such coverage by collecting a wide range of information including kernel activity, CUDA API calls, memory allocations, Python call stacks, and even programmable kernel-level measurements, but at substantial runtime and data-volume cost [17, 19]. More critically, this overhead induces an observer effect, directly slowing training and potentially distorting the observed behavior.

Challenge 2: Real-time analysis over massive distributed traces. Even with low-overhead collection, kernel traces at 10,000-GPU scale still produce enormous event volumes. In a 10,000-GPU training job, each GPU generates 10^4 – 10^5 kernel events per minute, producing 6–60 GB/min of raw traces cluster-wide. Performing cross-rank comparison directly on full raw data is infeasible in both computation and storage; yet over-aggregating would lose the information needed to pinpoint anomalous kernels.

These challenges show that ARGUS is not enabling a lighter-weight profiler, but requires rethinking observation scope, data representation, and diagnosis workflow.

2.4 Design Space Under Production Constraints

The resolution of these challenges does not admit a single optimal solution, but rather involves tradeoff choices.

Observation scope—coverage vs. perturbation. At one end of the design space lies the comprehensive view of a single profiler (strong diagnostic capability but cannot be always-on); at the other end lies low-overhead monitoring of coarse-grained metrics (can be always-on but diagnostically insufficient). ARGUS chooses a middle path: decomposing observation by training execution hierarchy, collecting CPU call stack, framework semantics, and kernel activity as complementary signals, each responsible for a single type of information with bounded overhead (detailed in §4).

Trace representation—fidelity vs. scalability. Raw traces offer the highest fidelity but cannot be directly used for online transport, storage, and cross-rank analysis at 10,000-GPU scale in production; pure metrification is insufficient to localize specific kernels responsible for anomalies. ARGUS chooses tiered data representation: the online path uses structured metrics and kernel statistical summaries to support real-time queries and anomaly detection, while complete Perfetto traces are persisted to object storage for deep analysis of anomalous windows (detailed in §5).

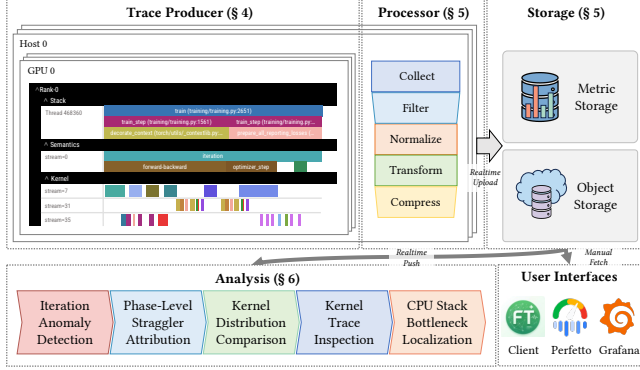


Figure 3. Overall architecture of ARGUS.

Diagnosis workflow—speed vs. depth. Performing fine-grained search across all ranks, all kernels, and all time windows is prohibitively expensive; outputting only anomalous time intervals cannot guide remediation. ARGUS chooses progressive diagnosis: multiple detection levels run in parallel, each covering a different granularity—from detecting anomalous time periods, to simultaneously localizing straggler ranks and bottleneck phases, to identifying anomalous kernels. Finally, deep-dive confirmation is performed on a small number of ranks and windows (detailed in §6).

These three design choices jointly shape ARGUS’s architecture: low-overhead runtime monitoring (§4), scalable trace processing (§5), and progressive diagnosis (§6).

3 System Overview

3.1 Architecture and Data Flow

Trace Producer (§4). Deployed within each training process, continuously producing three complementary observations with low overhead. CPU call stack profiling captures Python call stacks via external sampling; framework semantics instrumentation records the GPU-side duration of each training phase; kernel execution tracing continuously records every kernel’s launch time, duration, and stream.

Processor (§5). An independent process deployed per host, receiving raw event streams from all local Trace Producers. It filters, normalizes, and converts events into Perfetto [15] format, and performs online statistical compression on the kernel trace, condensing kernel events within each time window into KB-scale structured statistical summaries.

Storage (§5). Tiered storage. Metric Storage ingests structured metrics and kernel statistical summaries, supporting Grafana [16] visualization and low-latency alerting in real time. Object Storage persists complete Perfetto trace files.

Analysis (§6). Implements automated detection algorithms that identify anomalous windows, straggler ranks, and degraded kernels, and determines the scope for confirmation.

3.2 User Interfaces

ARGUS’s user-facing entry point is FT-Client, a unified diagnostic interface integrating real-time monitoring and deep

analysis capabilities. After specifying a training job and time range, FT-Client presents results through two visualization systems: the Grafana Dashboard displays per-rank iteration time, phase durations, kernel anomaly alerts, and cross-rank comparisons in real-time. Perfetto loads execution traces from object storage for deep-dive analysis, presenting kernel execution timing, semantic phase durations, and CPU call stacks in a unified timeline view.

4 Low-overhead Runtime Monitoring

This section instantiates the observation-scope choice from §2.4, decomposing observation into three hierarchy-specific signals. ARGUS does not use a single profiling tool to comprehensively capture all information; instead, each signal targets a specific layer of the execution hierarchy. The three collection channels are complementary rather than substitutive; starting or stopping any one does not affect the others.

4.1 CPU Stack Sampling

Fail-slow causes do not always originate from the GPU. Python GC pauses, data loading stalls, CPU contention, and GIL contention can equally lead to iteration time anomalies. ARGUS employs py-spy [14] to obtain Python call stacks by reading the target process’s memory, requiring no modifications to training code or injection of hooks. We adapt it for streaming: instead of generating a single flamegraph, it continuously outputs structured call stack snapshots in fixed sampling windows, giving CPU-side observation the same temporal continuity as GPU-side traces. When semantics shows an anomalously long phase but kernel execution tracing indicates normal GPU execution time, the CPU call stack from the same window can quickly identify host-side stalls (such as GIL contention, data preprocessing bottlenecks, or anomalous system calls), avoiding misattribution of non-GPU problems to kernel or communication issues.

4.2 Framework Semantics

ARGUS inserts CUDA Events at the entry and exit of key framework phases (forward, backward, optimizer, communication), using the elapsed time between two events on the same CUDA stream to capture the true GPU-side execution duration of that semantic interval [28]. Unlike CPU-side wall-clock time, a CUDA Event is timestamped only when the GPU actually reaches that position in the stream’s execution order, thus more accurately reflecting device-side execution time and being less affected by CPU scheduling, asynchronous submission, and host/device decoupling. The instrumentation does not modify the framework’s internal implementation or optimizer logic; it performs lightweight wrapping only at call sites where semantic boundaries are clear, concentrating profiling logic on a few critical paths.

A key challenge is accurately identifying the CUDA stream on which the target phase actually executes. Computation

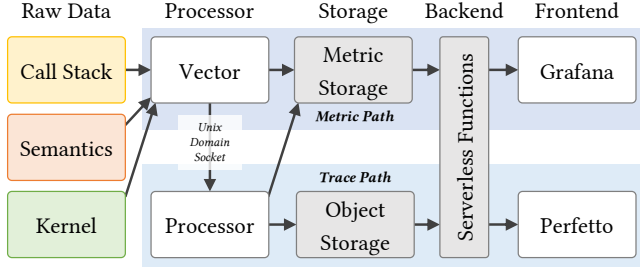


Figure 4. Data pipeline architecture.

operations typically run on the default stream, but communication operations use NCCL [30] internal streams. If events are inserted on the default stream, communication duration is near-zero. Therefore, the actual execution stream must be determined based on communication type: collective communication selects the stream based on device, P2P communication based on peer group rank [32].

4.3 Kernel Activity

ARGUS continuously records GPU kernel names, launch times, durations, and stream assignments via the CUPTI Activity API [29], injecting into the training process via environment variables without requiring modifications to the training framework or user code. To avoid the callback path becoming a bottleneck on the training hot path, ARGUS organizes the tracing backend into three decoupled paths: the *control path* only transmits start/stop signals; the *collection path* performs only the most lightweight operation in the callback—receiving the buffer and handing it off to the backend via a queue; the *processing and export path* asynchronously completes parsing, format conversion, and disk writes in an independent thread without blocking the frontend collection. Beyond this architecture design, ARGUS further controls overhead and stability through selective process injection, pre-allocated buffer reuse, and bounded resources with backpressure (detailed in Appendix A).

5 Scalable Trace Processing

This section instantiates the trace-representation choice from §2.4, transforming heterogeneous traces into scalable online summaries and complete persisted traces.

5.1 Data Pipeline Architecture

ARGUS interposes a unified data pipeline between runtime monitoring and progressive diagnosis. The ingestion and transformation layer is implemented using Vector [8] for its high-throughput and lightweight transformation capabilities. As shown in Figure 4, Vector ingests three types of local observation logs and distributes them to two downstream paths based on their characteristics. The **trace path** forwards raw event streams via Unix domain socket to an independent Processor process. The Processor is responsible for two tasks. First, it filters, normalizes, and converts raw events into Perfetto format, writing them to Object Storage



Figure 5. Repetitive kernel execution patterns.

for subsequent deep analysis. Second, it performs online statistical compression on kernel traces (§5.2), writing compressed structured summaries to Metric Storage for real-time cross-rank comparison. The **metrics path** handles directly quantifiable observation results (phase duration, iteration time), writing them to Metric Storage via the Prometheus Remote Write protocol [4] to support real-time Grafana visualization and low-latency alerting.

This tiered design separates metrics for real-time monitoring from traces that require preservation of complete temporal semantics, and decouples lightweight data ingestion from the heavier trace materialization process, ensuring that neither process becomes a bottleneck for the other.

5.2 Online Statistical Compression of Kernel Traces

In a 10,000-GPU cluster, each GPU can produce 10^4 – 10^5 kernel events per minute, and the total raw trace volume per hour can reach hundreds of gigabytes. Performing cross-rank anomaly analysis directly on the full raw data is infeasible in both computation and storage. To address this, ARGUS designs an online statistical compression method within the Processor, transforming raw kernel events into compact structured summaries that enable subsequent cross-rank anomaly detection (§6.2) on KB-scale data.

Rationale for statistical compression. As shown in Figure 5, kernel execution in distributed training exhibits highly regular repetitive patterns: each stream on the same rank repeatedly executes the same combination of kernels in a fixed sequence (e.g., stream=7 exhibits a SendRecv–AllGather–SendRecv alternating sequence), and the kernel durations at the same position across repetitions is highly consistent. Furthermore, ranks with the same parallel role execute identical kernel sequences, so under normal conditions the duration distribution of the same kernel across ranks should exhibit highly consistent statistical characteristics. This regularity makes it possible to extract statistics for each kernel and perform cross-rank comparison.

However, directly computing a single statistic across all duration samples of the same kernel is problematic. Although kernels of the same name at the same position have highly consistent durations, kernels of the same name at different positions may differ significantly: as shown in Figure 5 for stream=7, the first SendRecv and the third SendRecv on the same stream differ in duration by several multiples due to different data transfer volumes; AllGather kernels on different streams (e.g., stream=7 vs. stream=31) differ even more dramatically in time scale due to participation in different communication groups. This multimodal structure means that computing a global median without distinction would

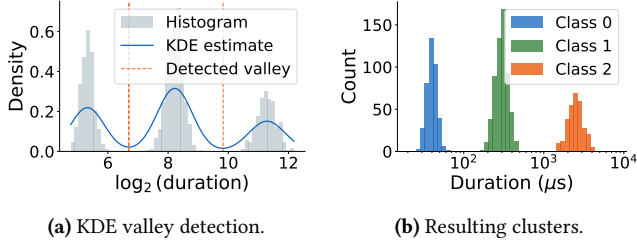


Figure 6. Visualization of KDE-based clustering.

mask positional and stream-level differences, leading to false positives in anomaly detection. Therefore, ARGUS first clusters kernel durations to identify each mode, and then extracts statistics separately for each mode.

Clustering method. The online clustering algorithm must satisfy three constraints. First, it must not require pre-specifying the number of clusters K , since the number of modes varies greatly across different kernels. Second, it must not require historical data or a warm-up phase, as the system should be able to run independently on any time window. Third, its time complexity must be linear in the number of samples, as online deployment requires computation to complete by the end of each window. Based on these requirements, ARGUS adopts the valley detection method based on kernel density estimation (KDE): it constructs a density curve over log-duration samples and uses local minima (valleys) as cluster boundaries, with the number of clusters automatically determined by the data’s modal structure.

Algorithm procedure. For each $(kernel, stream, rank)$ combination in each time window, the algorithm first applies a logarithmic transformation to the raw durations, then computes the KDE density function on an equally-spaced grid for subsequent valley detection:

$$\hat{f}(x) = \frac{1}{n} \sum_{i=1}^n K\left(\frac{x - x_i}{h}\right) \quad (1)$$

where $K(\cdot)$ is the Gaussian kernel function [36] and the bandwidth h is automatically determined by Scott’s rule [36]: $h = 1.06 \cdot \sigma \cdot n^{-1/5}$. All local minima on the density curve are identified as candidate cluster boundaries, and two layers of filtering are applied to eliminate noise: *cluster-level filtering* requires that both sides of each valley contain sufficient samples, preventing noise fluctuations from being misidentified as independent modes; *spacing filtering* requires that the duration difference between adjacent retained boundaries is sufficiently significant, preventing pseudo-valleys within the same peak from causing over-segmentation. Figure 6 visualizes this process. In log-space, the KDE density curve clearly exhibits multi-peak structure, and the algorithm determines cluster boundaries at density minima (panel (a)). Mapping these boundaries back to linear duration space partitions the original samples into discrete clusters, each corresponding to a characteristic execution time scale (panel (b)). Finally, three statistics are computed for each cluster: execution count

Table 2. Overview of ARGUS diagnostic levels.

Level	Data Source	Mode	Purpose	Latency
L1	Iteration time series	Auto	Anomalous time window classification	Seconds
L2	Semantic phase durations	Auto	Straggler rank and bottleneck phase	Seconds
L3	Kernel statistical summaries	Auto	Degraded kernel identification	Minutes
L4	Execution trace	Manual	Critical path and root cause confirmation	On-demand
L5	CPU call stacks	Manual	Host-side stall localization	On-demand

count, median duration *p50*, and 99th percentile duration *p99*.

Compression effectiveness. Through the above method, all raw events for each $(kernel, stream, rank)$ within a time window are compressed into a few cluster triples $(count, p50, p99)$. Taking 10,000-GPU scale as an example, if each rank has approximately 100 active $(kernel, stream)$ combinations with an average of 2 clusters each, then each rank needs to upload only about 200 statistical records per minute (~several KB), and the total volume from 10,000 ranks is approximately tens of MB; compared to the hundreds of GB of raw traces, this achieves a compression ratio of $10^3 - 10^4$. This makes real-time comparative analysis across all ranks feasible in both computation and storage.

The compression is lossy but fidelity-preserving: *p50* captures the typical execution time of each kernel class, *p99* retains tail latency, and *count* records the frequency proportion of each cluster. As described in §6.2, these three statistics are sufficient to support cross-rank anomaly detection via CDF reconstruction and Wasserstein distance [40]. The entire statistical compression process is performed within the Processor, running asynchronously from the training process without blocking the training main loop.

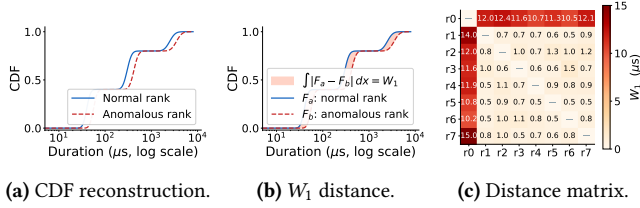
6 Progressive Diagnosis

Building on the observation-scope and trace-representation choices from §2.4, this section instantiates the diagnosis-workflow choice, progressively narrowing the search space.

ARGUS’s diagnostic framework consists of five levels, with L1, L2, and L3 running in parallel as three automated levels covering different granularities (iteration time / semantic phase / kernel statistics), jointly narrowing the scope requiring manual analysis. L1 serves fail-slow detection by identifying anomalous time windows. L2 and L3 serve both fail-slow localization and performance optimization: they pinpoint straggler ranks and degraded kernels, while also revealing inefficiencies such as load imbalance or suboptimal communication overlap. L4/L5 provide high-fidelity deep-dive confirmation primarily for performance optimization, enabling engineers to inspect execution traces for root-cause analysis and optimization opportunities, as well as offline critical path analysis and host-side idle-cause localization. At 10,000-GPU scale, performing fine-grained analysis on all ranks is neither economical nor necessary; the three automated levels reduce the search scope from tens of thousands of ranks to single-digit ranks and time windows.

Table 3. Representative parallelism-group-aware routing rules.

Event Type	Comparison Group
gated_mla_self_attention, self_attention	DP group
moe_layer, moe_experts	EP group
dp-allreduce, dp-reduce-scatter	DP group
ep-alltoall	EP group
...	...

**Figure 7.** Kernel statistics anomaly detection workflow.

6.1 Iteration-level Detection (L1) and Phase-level Attribution (L2)

L1 continuously collects each rank’s iteration time series, running two complementary anomaly detection algorithms: sliding-window ratio-gated jitter detection for short-term fluctuations and spikes, while full-scan change-point detection for step-wise regression. Together they classify iteration time behavior as stable, jitter, regression, or both.

L2 performs cross-rank comparison on the phase durations recorded by framework semantics instrumentation within parallelism groups, using the coefficient of variation (CV) to quantify intra-group inconsistency and z-scores to identify straggler ranks. Its key design is **parallelism-group-aware routing**: since distributed training uses multiple parallelism dimensions simultaneously [27, 34, 37, 44, 45], each event must be compared only among ranks that share the same parallel role. The system maintains a routing table that maps each semantics event to its corresponding parallelism group, as illustrated in Table 3. For pure computation events such as `self_attention` and `moe_experts`, a high CV within the corresponding group directly indicates a straggler rank whose compute is slower. For communication events such as `dp-allreduce` and `ep-alltoall`, the system additionally determines whether the prolonged duration originates from the rank itself or waiting for a slow peer in the same synchronization group. The complete algorithm descriptions for L1 and L2 are provided in Appendix B.

6.2 Kernel Statistics Anomaly Detection (L3)

Based on the compressed statistics described in §5.2, ARGUS designs a cross-rank anomaly detection method that identifies performance anomalies by comparing the execution time distribution of the same kernel across different ranks. The core idea is that in synchronous distributed training, ranks with the same parallel role should execute identical kernel sequences, and their duration distributions should be highly consistent. When the distribution of a specific kernel on a particular rank significantly deviates from other ranks, that rank is identified as a fail-slow suspect. Unlike

the phase-duration-based cross-rank comparison in §6.1, this level of analysis drills down to individual kernel granularity, precisely identifying which specific kernel is behaving anomalously, and providing a precise entry point for subsequent root-cause confirmation via Perfetto traces.

Figure 7 illustrates the detection workflow: (a) CDFs are reconstructed from compressed statistics for each rank, where the anomalous rank’s CDF curve clearly deviates from normal ranks; (b) the Wasserstein-1 distance between any two ranks’ reconstructed CDFs quantifies distribution difference; (c) the W_1 values for all rank pairs form a matrix, where the anomalous rank shows systematically elevated distances to others, enabling accurate identification.

CDF reconstruction. At 10,000-GPU scale, directly performing cross-rank comparison analysis on full raw samples is infeasible; therefore, ARGUS reconstructs cumulative distribution functions (CDFs) from the compressed statistics obtained in §5.2 for distribution comparison. ARGUS employs a parametric reconstruction method based on the log-normal assumption: for each cluster c , a log-normal component is constructed with location parameter $\mu_c = \ln(p_{50_c})$, positioning the distribution center at the median, and scale parameter $\sigma_c = \frac{\ln(p_{99_c}) - \ln(p_{50_c})}{2.326}$, fitting the tail shape using the relationship with the standard normal distribution’s 99th percentile point $z_{0.99} = 2.326$. The components are weighted by *count* to form a mixture CDF:

$$F(x) = \sum_c \frac{\text{count}_c}{\sum_{c'} \text{count}_{c'}} \cdot \Phi\left(\frac{\ln x - \mu_c}{\sigma_c}\right) \quad (2)$$

where $\Phi(\cdot)$ is the CDF of the standard normal distribution. This reconstruction method utilizes median and tail information, capturing both the overall shape and tail latency characteristics of the distribution. The log-normal assumption is chosen because kernel durations in practice exhibit a clearly right-skewed distribution that approximates normality after log-transformation, allowing each mode’s shape to be well parameterized using only two percentiles (p_{50} , p_{99}).

Wasserstein-1 distance and anomaly identification. From the reconstructed CDFs, ARGUS uses the Wasserstein-1 distance (W_1 , also known as Earth Mover’s Distance) to quantify distribution differences between two ranks:

$$W_1(F_a, F_b) = \int_0^\infty |F_a(x) - F_b(x)| dx \quad (3)$$

This distance is computed via trapezoidal numerical integration on a log-uniform grid. The choice of W_1 over KL divergence or the KS statistic is motivated by the following: W_1 possesses true metric properties (satisfying the triangle inequality), is sensitive to both shifts and scaling of distributions, and has an intuitive physical interpretation—it can be understood as the minimum work required to “transport” one distribution into another [40]. For fail-slow detection scenarios, W_1 can simultaneously capture overall distribution shift (persistent slowdown) and tail inflation (intermittent

tail latency), whereas the KS statistic only considers the maximum deviation point, and KL divergence is unstable when distribution supports do not completely overlap.

For each (*kernel*, *stream*) combination, the system computes the W_1 values between all pairs of ranks and organizes them into a distance matrix. Under normal conditions, ranks with the same parallel role execute identical kernel sequences, and pairwise W_1 values should be relatively small. If a rank exhibits a performance anomaly, its W_1 with all other ranks will be systematically elevated, manifesting as significantly higher values in the corresponding row and column of the distance matrix compared to other positions.

Cross-rank IQR anomaly determination. To automatically identify anomalous ranks from the distance matrix, ARGUS computes the mean W_1 of each rank to all other ranks as that rank’s deviation score, then applies a robust statistical method based on the interquartile range (IQR) for outlier-based anomaly determination [39]:

$$\text{upper_fence} = Q_3 + \alpha \cdot (Q_3 - Q_1) \quad (4)$$

where Q_1 and Q_3 are the 25th and 75th percentiles of all ranks’ deviation scores, respectively, and α is the anomaly coefficient. Ranks exceeding the upper fence are flagged as anomalous. The IQR method is chosen over mean-and-standard-deviation-based determination because IQR is robust to extreme values: even if some ranks exhibit extreme deviations, the estimates of Q_1 and Q_3 are not distorted, thus not affecting the baseline for judging other ranks.

6.3 Deep-dive Confirmation (L4/L5)

L4/L5 do not perform 10,000-GPU-wide searches; instead, they provide high-fidelity root-cause confirmation for the small number of ranks and anomalous windows identified by L1–L3. Grafana duration scatter plots reveal how a specific kernel’s duration evolves over time, and heatmaps display anomaly distribution across a rank-by-time matrix. For finer-grained inspection, engineers examine the Perfetto timeline to inspect kernel execution timing, semantic phase durations, and CPU call stacks in a unified view. Beyond interactive inspection, L4 also supports offline critical path analysis using approaches similar to Holistic Trace Analysis [25], identifying the longest sequential dependency chain that determines iteration time. L5 performs offline analysis of CPU call stacks to localize host-side causes when both compute and communication are simultaneously idle, pinpointing which function is the contributor to the stall.

7 Implementation

We implement the runtime monitoring components of ARGUS in approximately 2.7K lines of C++, 16.7K lines of Python, and a streaming adaptation of py-spy. The kernel execution tracing component is compiled as a standalone shared library (`libcupti_injector.so`) and injected into

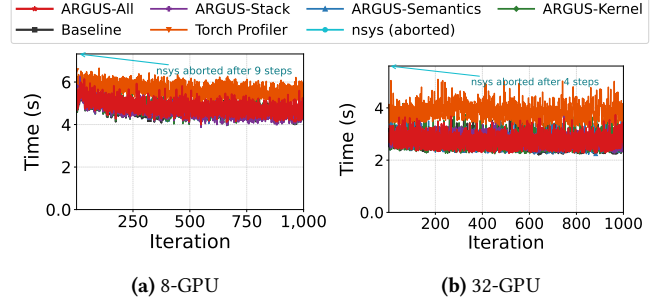


Figure 8. Training time under different profiling configurations.

the target training process via `CUDA_INJECTION64_PATH` environment variable. The CUDA runtime automatically loads this library during initialization, requiring no modifications to the training code or launch scripts. Framework semantics instrumentation is provided as a Python package. The training framework enables it by calling the exposed API at critical path boundaries, while the underlying CUDA Event management and NCCL stream queries are implemented as C++ extensions. The Processor is implemented in Go (approximately 7.3K lines) and receives raw traces over Unix domain sockets, performing Perfetto encoding, online kernel statistical compression, and storage writes. The diagnosis and analysis service is implemented in Python (approximately 24K lines) and encapsulates the three-level automated detection algorithms described in §6.

All components are deployed as sidecars alongside training tasks and enabled via a single environment variable, supporting batch onboarding across 10,000-GPU scale clusters.

8 Evaluation

This section evaluates ARGUS along two dimensions: runtime overhead (§ 8.2) and data volume and compression characteristics at scale (§ 8.3). As a complementary evaluation, fault diagnosis capability is evaluated in Appendix D.

8.1 Experimental Setup

Experiments are conducted on an 8-GPU node with intra-node GPUs interconnected via NVLink. We train the HunYuan-V3 Preview model [38] (configuration details in Appendix C). We compare ARGUS against two widely-used profiling tools: (1) **PyTorch Profiler**, the built-in performance analysis tool in PyTorch [33], and (2) **NVIDIA Nsight Systems**, a system-level profiling tool. All tools are configured in always-on mode throughout training.

8.2 Runtime Overhead

Iteration time. As shown in Figure 8, we train for 1,000 iterations at both 8-GPU and 32-GPU scales and compare per-iteration time across configurations. Among ARGUS components, semantics instrumentation and stack sampling introduce negligible overhead, kernel execution tracing (CUPTI)

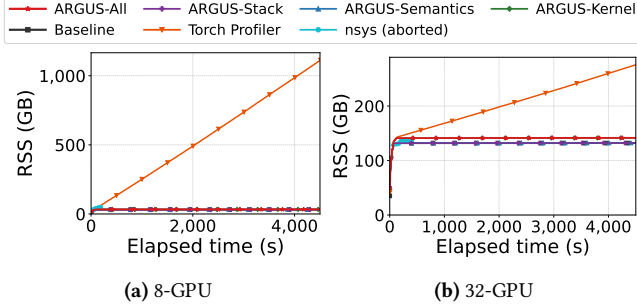


Figure 9. Resident Set Size (RSS) over time.

adds approximately 1%–2%, and all three combined remain within 2%. As model size grows and GPU computation dominates, this overhead further diminishes. In contrast, PyTorch Profiler inflates iteration time by 20%–44% and eventually triggers out-of-memory failures due to unbounded trace accumulation, making it entirely impractical for production use. nsys fails to complete training under always-on mode at either scale: at 8-GPU, training produces NaN at iteration 10; at 32-GPU, training hangs during AllToAll communication. Neither failure occurs in the baseline, confirming the observer-effect challenge discussed in § 2.3: general-purpose profilers can distort or break training execution.

Memory footprint. As shown in Figure 9, we continuously monitor RSS across all configurations. ARGUS adds approximately 2 GB (8-GPU) and 10 GB (32-GPU) over the baseline, primarily from the pre-allocated CUPTI ring buffer; semantics instrumentation and stack sampling consume negligible additional memory. Because ARGUS employs a streaming architecture that hands off collected data to the Processor immediately without accumulating raw traces locally, its memory footprint remains constant regardless of training duration. In contrast, PyTorch Profiler accumulates complete trace data in memory until the profiling window ends, causing RSS to grow continuously until OOM. nsys exhibits similar trace-buffer inflation.

In summary, ARGUS achieves a total overhead of less than 2% with constant memory overhead when all three observation sources are active, enabling continuous always-on operation in production environments. PyTorch Profiler and nsys are unsuitable as always-on observability solutions due to overhead or training-breaking side effects.

8.3 Data Volume and Compression

This experiment evaluates the per-rank per-step data volume generated under each profiling configuration, and the compression effectiveness of the Processor pipeline, including KDE-based kernel clustering component (§ 5.2).

Table 4 summarizes the data volume at each processing stage. ARGUS’s three observation sources collectively produce approximately 10.6 MB of raw data per rank per step. After the Processor converts this into Perfetto-format traces, the volume compresses to 443 KB. The structured summaries

Table 4. Per-rank per-step data volume at each processing stage.

Source	Raw	Perfetto trace	Metric Storage
CPU call stack (py-spy)	250 KB	8 KB	—
Kernel execution (CUPTI)	10 MB	420 KB	2.7 KB
Framework semantics	394 KB	15 KB	16 KB
ARGUS total	10.6 MB	443 KB	18.7 KB
nsys (baseline)	63.56 MB	—	—
PyTorch Profiler (baseline)	48.76 MB	—	—

uploaded to Metric Storage total only 18.7 KB, of which kernel data achieves a compression ratio of approximately 3,700× through clustering (from 10 MB to 2.7 KB).

At 10,000-GPU scale with approximately 15 steps per minute, the aggregate upload rate is $10,000 \times 15 \times 18.7 \text{ KB} \approx 2.7 \text{ GB/min}$, within time-series database capacity and enabling real-time cross-rank analysis. In contrast, nsys and PyTorch Profiler generate 63.56 MB and 48.76 MB per rank per step respectively. At 10,000-GPU scale, a single step produces over 600 GB (nsys) or 470 GB (PyTorch Profiler) of trace data, far exceeding the capacity of any online analysis system, and thus only suitable for offline post-hoc analysis.

9 Case Studies

This section presents five production case studies to demonstrate the practical effectiveness of ARGUS’s progressive diagnostic framework. The jobs use Megatron-LM-style hybrid parallelism. We describe each rank by its active parallel coordinates, including data parallelism (DP), tensor parallelism (TP), pipeline parallelism (PP), and expert parallelism (EP); unused dimensions are omitted. The cases cover compute straggler nodes (Case 1), communication link degradation (Case 2), pipeline bubble amplification (Case 3), operator JIT compilation (Case 4), and compute straggler with misleading out-of-band metrics (Case 5).

9.1 Case 1: Compute Straggler Localization

This case occurred in a ~4,000-GPU VLM training job with TP=2 and EP=8. L1 detected a persistent iteration-time regression: step time increased from about 4 s to more than 200 s for multiple consecutive steps. Concurrently, L2 performed CV analysis on semantics events (*self_attention*, *moe_experts*, *etc.*) across ranks within the DP group, detecting that the compute-class semantic events on ranks at DP=656 and DP=657 deviated significantly from the group mean, marking them as stragglers.

As shown in Figure 10, the Grafana dashboard visualizes the per-rank maximum operator duration as a heatmap, with the DP replica index on the x-axis and the TP index on the y-axis. Normal ranks spend 6–16 ms in *self_attention* and 4–6 ms in *mlp*. In contrast, both TP indices in DP replicas 656 and 657 are outliers: *self_attention* reaches 2,252 ms and 2,022 ms, while *mlp* reaches 480–726 ms. These phases are compute-only and involve no collective communication, so the anomaly points to local GPU compute degradation rather

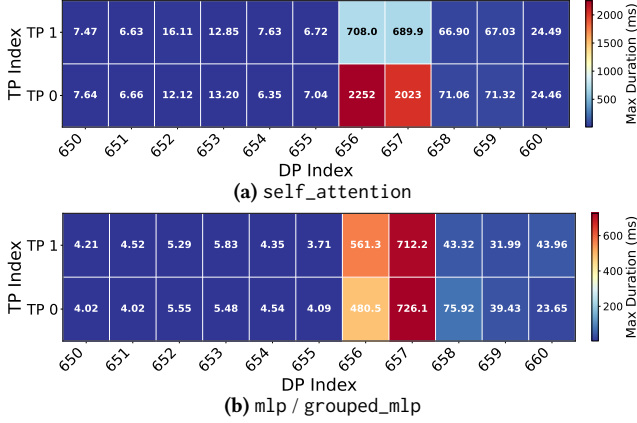


Figure 10. Case 1: Grafana heatmap of per-rank maximum operator duration. The x-axis is the DP replica index, and the y-axis is the TP index. DP replicas 656 and 657 are outliers across both TP indices, with more than 150 $\times$ degradation on compute-only operators.

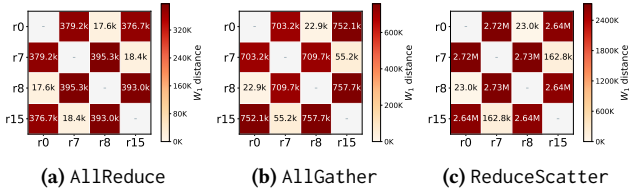


Figure 11. Case 2: W_1 distance matrices for three communication kernels. Ranks 0 and 8 belong to one EDP group; ranks 7 and 15 belong to another. Intra-group distances are small (17–23k), while inter-group distances are orders of magnitude larger (376k–2.73M), revealing systematic communication degradation in the EDP group containing ranks 7 and 15.

than synchronization delay. After excluding the affected nodes, training returned to its normal speed.

9.2 Case 2: Communication Link Degradation

This case occurred in a ~ 500 -GPU audio-model training job with $EP=8$. The iteration time was stable with no jitter or regression, but overall throughput remained persistently below the expected baseline. Neither L1 nor L2 detected any anomaly. L3 performed cross-rank W_1 comparison on the duration distributions of typical kernels on each rank, identifying significant cross-group distribution shifts on three communication kernel types and triggering an alert.

Figure 11 compares the pairwise W_1 distance matrix of four representative ranks from two expert data parallel (EDP) groups, with ranks 0 and 8 from one group and ranks 7 and 15 from another. The matrix exhibits a clear grouping pattern: intra-group W_1 distances are small (AllReduce: r0–r8 = 17.6k, r7–r15 = 18.4k), whereas cross-group distances are consistently larger (AllReduce: 376k–395k; AllGather: 703k–757k; ReduceScatter: 2.64M–2.73M). This indicates a systematic difference in communication kernel duration distributions between the EDP group containing ranks 7 and 15 and the group containing ranks 0 and 8.

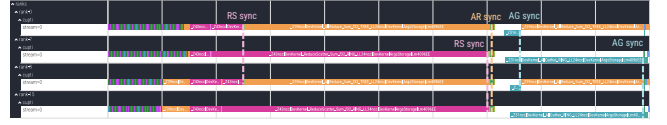


Figure 12. Case 2: L4 Perfetto trace of communication kernels. Rank 7 shows longer EDP-internal ReduceScatter and AllGather operations, illustrating network degradation in its own EDP group.

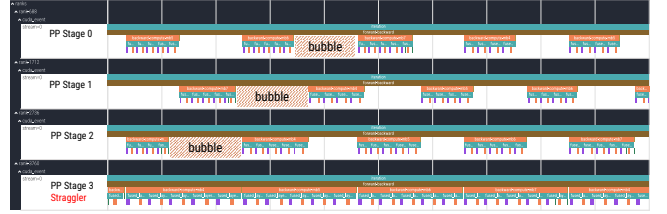


Figure 13. Case 3 (a): L4 Perfetto trace of one PP group (ranks 688/1712/2736/3760, PP stages 0–3). The last PP stage, rank 3760, has tightly packed backward-compute events and small bubbles. Upstream stages wait for gradients from the slow downstream stage.

We further verified this through the L4 Perfetto trace, as shown in Figure 12. The timeline reveals that rank 7’s EDP group executes AllGather and ReduceScatter (EDP-internal communication, synchronized only within the EDP group but not across EDP groups) significantly slower than other EDP groups, with no observable waiting time, indicating degradation originates from its own communication link rather than being slowed by other ranks. Conversely, other EDP groups (e.g., rank 0’s group) show prolonged AllReduce, because AllReduce must synchronize all EDP groups, and rank 7’s EDP group communication latency forces other groups to wait passively. Based on this analysis, we executed a targeted NCCL test on the nodes in the EDP group containing ranks 7 and 15, confirming that two machines in this group had PCIe link hardware faults. After repair, throughput recovered to the expected level. This silent degradation is undetectable by systems relying on iteration time thresholds or heartbeats (e.g., Greyhound [42], C4 [11]), because their detection logic depends on visible performance fluctuations as a trigger condition.

9.3 Case 3: Pipeline Bubble Amplification

This case occurred in a $\sim 4,000$ -GPU VLM job with $TP=4$, $PP=4$, and $EP=8$. The job ran slower than expected, yet none of L1, L2, or L3 triggered any automatic alert. We manually inspected per-rank semantics event durations through the Grafana Dashboard, observing that rank 3760’s backward-compute median duration was approximately 173 ms, while other ranks at the same PP index showed only about 92 ms — a ratio of about 1.9 $\times$. This prompted further investigation of rank 3760’s PP group.

We retrieved the L4 Perfetto trace and compared the 4 PP stages (ranks 688/1712/2736/3760) within the same PP group, as shown in Figure 13. The timeline exhibits a

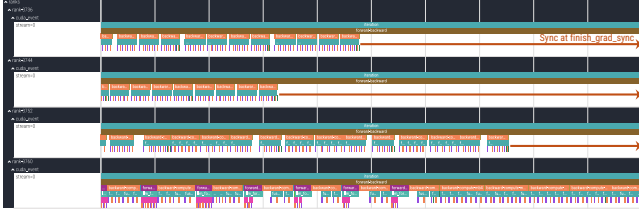


Figure 14. Case 3 (b): L4 Perfetto trace of ranks at the same PP index (PP stage 3) across different PP groups (ranks 3736/3744/3752/3760). Rank 3760’s backward-compute is denser; yet all ranks’ forward-backward durations are aligned at $\sim 11,178$ ms due to `finish_grad_sync`.

clear “asymmetric bubble” pattern: rank 3760 (the last PP stage) has tightly packed backward-compute events for each micro-batch, with average inter-event bubbles of only 160 ms; ranks 688/1712/2736 (the first three PP stages) show average bubbles of 227–233 ms. This indicates that rank 3760 is the compute bottleneck: its GPU runs at full load, filling bubbles with computation, while upstream PP stages wait for downstream gradient propagation, producing idle gaps.

We further compared ranks at the same PP index (PP stage 3) across different PP groups (ranks 3736/3744/3752/3760), as shown in Figure 14. The trace shows that rank 3760’s backward-compute events consistently finish later than its peers at the same PP stage, confirming it as the sole compute bottleneck. However, all ranks’ forward-backward total durations are nearly perfectly aligned (approximately 11,178 ms), because the trailing `finish_grad_sync` is a synchronous operation that forces all gradient aggregation to complete. This alignment effect causes the iteration time observed by L1/L2 to show minimal inter-rank differences, effectively masking the bottleneck. After removing the corresponding straggler node, training speed recovered to expectations. Existing systems (e.g., Holmes [43], Minder [9]) operate at iteration/operator or machine-level symptoms and do not model PP pipeline causal dependencies, making it difficult for them to identify straggler nodes concealed by synchronous alignment.

This case reveals two masking mechanisms for straggler nodes under PP parallelism [20, 24, 26]. First, pipeline dependency-induced “bubble transfer” causes the slow rank’s compute bottleneck to propagate through PP dependencies into idle gaps on other ranks, diffusing the anomaly across ranks rather than concentrating it at its source. Second, the alignment effect of `grad_sync` forces iteration time to converge across ranks through global synchronization, masking the performance difference. L1, L2, and L3 did not trigger alerts because the anomalous rank’s backward-compute peak duration is about $1.9\times$ that of normal ranks, while natural variation due to different image shapes in VLM tasks can reach $2\text{--}3\times$, and synchronous alignment further flattens the iteration time differences, preventing statistical tests from declaring the deviation significant.

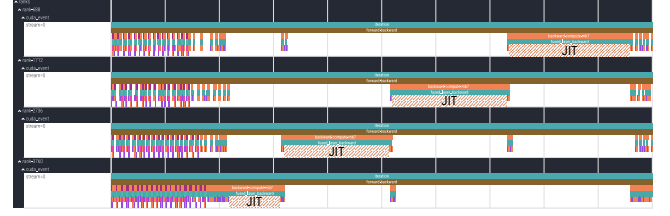


Figure 15. Case 4: L4 Perfetto trace of an anomalous step. In the PP group containing rank 688, backward-compute-mb7 becomes about $40\times$ longer than normal. Sparse kernel launches indicate host-side blocking rather than GPU computation.

9.4 Case 4: FlashAttention JIT Compilation

This case occurred in a $\sim 4,000$ -GPU VLM job with $TP=4$, $PP=4$, and $EP=8$. Training exhibited frequent iteration time spikes, with some steps experiencing latency surges of tens of times. L1 detected jitter and triggered an alert. However, L2 and L3 did not identify persistent anomalies because the spikes occurred only occasionally and recovered after a few steps, being diluted within their statistical windows.

Through L4 analysis in the Grafana dashboard, we observed a rare but extreme spike: the maximum backward-compute duration in the PP group containing rank 688 was about $40\times$ larger than that of other PP groups, although the spike was too short-lived to form a persistent L2 anomaly. We therefore retrieved the Perfetto trace for the anomalous step on rank 688, as shown in Figure 15, and compared ranks 688/1712/2736/3760 within the PP group. Within the anomalous step, most backward-compute-mb events take approximately 95–160 ms, but backward-compute-mb7 takes 6,303 ms (approximately $40\times$ inflation). The internal structure of this anomalous event shows only a single fused_layer_backward sub-event executing, with an extremely sparse kernel launch pattern where most time is consumed in host-side waiting rather than GPU computation. Combined with CUDA JIT compilation-related messages observed in the training logs, we confirmed the blocking originated from FlashAttention’s JIT compilation.

The recent FlashAttention implementation inherits the IO-aware exact-attention design [6], while its CuTe DSL backend [31] uses runtime JIT: the first invocation of an uncached kernel configuration lowers DSL code to PTX/cubin and caches the result [7]. Under the default configuration, compilation results are cached only in process memory; any fault-tolerance event (fatal node replacement, StepHang recovery, configuration hot-update) that restarts the process clears the cache. Since this job is a VLM with diverse input image shapes, JIT compilation is triggered frequently. The compilation-induced blocking propagates through PP dependencies: when compilation occurs on one rank, downstream PP ranks cannot receive activations or gradients, cascading the delay across the PP group.

The optimization measures adopted include: enabling disk caching via an environment variable so that compilation

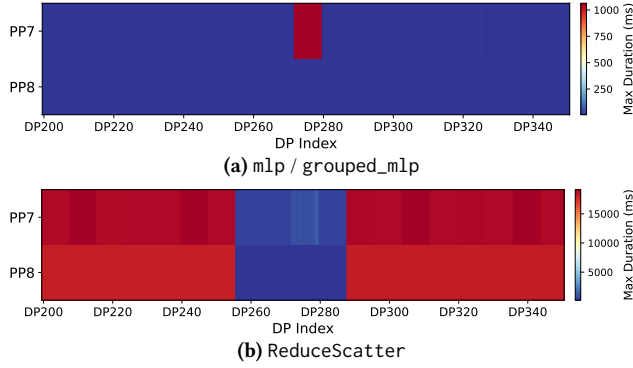


Figure 16. Case 5: Heatmap of per-rank max duration. The x-axis is the DP replica index, and the y-axis is the PP stage index. (a) MLP shows extreme degradation at PP=7, DP=272–279 (ranks 10352–10359, $\sim 5.7\times$). (b) The affected EP group spans DP replicas 256–287 and shows shorter ReduceScatter durations because compute stragglers delay its entry into DP-level communication.

artifacts persist across process restarts; and pre-enumerating possible shape combinations during a warm-up phase before training to complete compilation ahead of time. After these optimizations, JIT compilation spikes were completely eliminated. For such intermittent anomalies, traditional sampling-based profiling is unable to capture transient events. Even systems with always-on capability cannot trace an iteration-time spike to the specific operator-level blocking source without fine-grained execution traces that preserve per-event timing within each step.

9.5 Case 5: Compute Straggler with Misleading Out-of-band Metrics

This case occurred in a $\sim 10,000$ -GPU MoE training job with TP=1, PP=9, and EP=32. L1 detected iteration time regression from approximately 30 s to over 90 s, triggering an alert. L2 performed CV analysis on semantics events across ranks within the DP group. The compute-only mlp phase showed a CV above 0.8 on ranks 10352–10359 (PP=7, DP=272–279), whose durations were about $5.7\times$ the group mean.

As shown in Figure 16, the Grafana heatmap presents per-rank maximum operator duration with DP index on the x-axis and PP index on the y-axis. In Figure 16a, most ranks show stable mlp latency around 30–50 ms, while ranks 10352–10359 (PP=7, DP=272–279) reach 170–280 ms. Figure 16b reveals a complementary *inverse* pattern for ReduceScatter: looking vertically, *all* PP stages show *shorter* ReduceScatter durations than surrounding ranks. The 8 slow-compute ranks (10352–10359) reside in the same EP group as 24 other ranks (EP group: ranks 10336–10367, 32 ranks total). The EP-internal dispatch operation requires all 32 ranks to participate; the 8 slow ranks stall this collective, causing the entire EP group to enter the DP-level ReduceScatter late. As shown in the PP=7 row of Figure 16b, because the other EP groups in the same DP group are already ready when this EP group arrives, its ReduceScatter duration appears

shorter. Furthermore, PP inter-stage dependencies propagate the delay to all other PP stages in the same PP group: each stage must wait for the slow PP=7 stage to complete before proceeding, causing ranks across all PP stages at DP=256–287 (the full EP group’s DP range) to also enter their EP-group ReduceScatter late, exhibiting similarly shorter durations. This confirms compute degradation on ranks 10352–10359 as the root cause.

However, the out-of-band monitoring system reported a “server port down” event on one of the affected nodes, which operations initially attributed as the root cause and scheduled for network repair. ARGUS’s fine-grained tracing, in contrast, clearly showed that the anomaly manifested exclusively on pure-compute operators with no communication involvement, contradicting the network attribution. After replacing the affected nodes, training speed recovered immediately. This case demonstrates a key limitation of out-of-band metrics: communication anomalies observed at the infrastructure level do not necessarily originate from network hardware itself—they can be secondary effects of compute degradation propagating through collective dependencies. Without fine-grained visibility, operators risk misdiagnosing the problem and applying ineffective remediation.

Overall, fail-slow modes in such 10,000+ GPU training clusters are highly diverse, and no single diagnostic level can cover all scenarios. ARGUS follows a progressive workflow: L1–L3 automatically narrow the scope, while L4/L5 provide on-demand root-cause confirmation, completing the loop from anomaly discovery to localization.

10 Discussion

Agent-based intelligent diagnosis. The current framework achieves automated anomaly detection and preliminary attribution, but the complete closed loop from detection to root-cause confirmation still relies on engineer expertise. We are exploring integrating LLM agents that take ARGUS’s detection results as input, combine them with parallel topology and historical fault patterns, and automatically perform multi-round reasoning and tool invocations to produce structured diagnostic reports. Preliminary experience shows that agents can compress average diagnosis time from tens of minutes to the order of minutes, reducing the reliance on manual on-call intervention.

Low invasiveness and generalization. Among the three observation mechanisms, kernel execution tracing is injected via environment variables, and CPU call-stack profiling operates through external process sampling; neither requires any modification to training code. Framework semantics instrumentation only adds minimal instrumentation at critical paths. This low invasiveness makes ARGUS applicable beyond pre-training. In practice, ARGUS has already been extended to reinforcement learning training, and we plan to generalize it to inference serving in the future.

11 Conclusion

This paper presents ARGUS, a real-time, low-overhead, fine-grained tracing and analysis system for 10,000-GPU scale training clusters. ARGUS decomposes observation into three independently optimized mechanisms, achieving always-on tracing with less than 2% overhead. The unified data pipeline handles heterogeneous trace ingestion, tiered storage, and online statistical compression, transforming voluminous raw traces into KB-scale structured summaries for real-time cross-rank analysis while persisting complete traces for deep-dive inspection. On top of this, the progressive diagnosis framework narrows the diagnostic scope from tens of thousands of ranks to single-digit suspects. ARGUS has been deployed on a 10,000+ GPU production cluster for over six months, running stably alongside production training and playing a key role in rapid fail-slow detection and performance optimization.

References

- [1] AI at Meta. 2024. The Llama 3 Herd of Models. *arXiv preprint arXiv:2407.21783* (2024).
- [2] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language Models are Few-Shot Learners. In *Advances in Neural Information Processing Systems 33 (NeurIPS)*.
- [3] Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, Parker Schuh, Kensen Shi, Sasha Tsvyashchenko, Joshua Maynez, Abhishek Rao, Parker Barnes, Yi Tay, Noam Shazeer, Vinodkumar Prabhakaran, Emily Reif, Nan Du, Ben Hutchinson, Reiner Pope, James Bradbury, Jacob Austin, Michael Isard, Guy Gur-Ari, Pengcheng Yin, Toju Duke, Anselm Levskaya, Sanjay Ghemawat, Sunipa Dev, Henryk Michalewski, Xavier Garcia, Vedant Misra, Kevin Robinson, Liam Fedus, Denny Zhou, Daphne Ippolito, David Luan, Hyeontaek Lim, Barret Zoph, Alexander Spiridonov, Ryan Sepassi, David Dohan, Shivani Agrawal, Mark Omernick, Andrew M. Dai, Thanumalayan Sankaranarayanan Pillai, Marie Pellat, Aitor Lewkowycz, Erica Moreira, Rewon Child, Oleksandr Polozov, Katherine Lee, Zongwei Zhou, Xuezhi Wang, Brennan Saeta, Mark Diaz, Orhan Firat, Michele Catasta, Jason Wei, Kathy Meier-Hellstern, Douglas Eck, Jeff Dean, Slav Petrov, and Noah Fiedel. 2023. PaLM: Scaling Language Modeling with Pathways. *Journal of Machine Learning Research* 24, 240 (2023), 1–113.
- [4] Cloud Native Computing Foundation. 2024. Prometheus: Monitoring system and time series database. <https://prometheus.io>.
- [5] Weihao Cui, Ji Zhang, Han Zhao, Chao Liu, Jian Sha, Bo Sang, Bingsheng He, Minyi Guo, and Quan Chen. 2026. FLARE: Anomaly Diagnostics for Divergent LLM Training in GPU Clusters of Thousand-Plus Scale. In *23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI 26)*. USENIX Association, Renton, WA, 1021–1035.
- [6] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. In *Advances in Neural Information Processing Systems 35 (NeurIPS)*.
- [7] Dao-AILab. 2025. FlashAttention-4: CuTe DSL JIT Compilation and Caching. <https://github.com/Dao-AILab/flash-attention>.
- [8] Datadog. 2024. Vector: A lightweight, ultra-fast tool for building observability pipelines. <https://vector.dev>.
- [9] Yangtao Deng, Xiang Shi, Zhuo Jiang, Xingjian Zhang, Lei Zhang, Zhang Zhang, Bo Li, Zuquan Song, Hang Zhu, Gaohong Liu, Fuliang Li, Shuguang Wang, Haibin Lin, Jianxi Ye, and Minlan Yu. 2025. Minder: Faulty Machine Detection for Large-scale Distributed Model Training. In *22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI 25)*. USENIX Association, Philadelphia, PA, 505–521.
- [10] Yangtao Deng, Lei Zhang, Qinlong Wang, Xiaoyun Zhi, Xinlei Zhang, Zhuo Jiang, Haohan Xu, Lei Wang, Zuquan Song, Gaohong Liu, Yang Bai, Shuguang Wang, Wencong Xiao, Jianxi Ye, Minlan Yu, and Hong Xu. 2025. Mycroft: Tracing Dependencies in Collective Communication Towards Reliable LLM Training. In *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles (SOSP '25)*. Association for Computing Machinery, Seoul, Republic of Korea, 254–269.
- [11] Jianbo Dong, Bin Luo, Jun Zhang, Pengcheng Zhang, Fei Feng, Yikai Zhu, Ang Liu, Zian Chen, Yi Shi, Hairong Jiao, Gang Lu, Yu Guan, Ennan Zhai, Wencong Xiao, Hanyu Zhao, Man Yuan, Siran Yang, Xiang Li, Jiamang Wang, Rui Men, Jianwei Zhang, Chang Zhou, Dennis Cai, Yuan Xie, and Binzhang Fu. 2025. Enhancing Large-Scale AI Training Efficiency: The C4 Solution for Real-Time Anomaly Detection and Communication Optimization. In *2025 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 1246–1258.
- [12] Jianbo Dong, Kun Qian, Pengcheng Zhang, Zhilong Zheng, Liang Chen, Fei Feng, Yichi Xu, Yikai Zhu, Gang Lu, Xue Li, Zhihui Ren, Zhicheng Wang, Bin Luo, Peng Zhang, Yang Liu, Yanqing Chen, Yu Guan, Weicheng Wang, Chaojie Yang, Yang Zhang, Man Yuan, Hanyu Zhao, Yong Li, Zihan Zhao, Shan Li, Xianlong Zeng, Zhiping Yao, Binzhang Fu, Ennan Zhai, Wei Lin, Chao Wang, and Dennis Cai. 2025. Evolution of Aegis: Fault Diagnosis for AI Model Training Service in Production. In *22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI 25)*. USENIX Association, Philadelphia, PA, 865–881.
- [13] William Fedus, Barret Zoph, and Noam Shazeer. 2022. Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity. *Journal of Machine Learning Research* 23, 120 (2022), 1–39.
- [14] Ben Frederickson. 2024. py-spy: Sampling profiler for Python programs. <https://github.com/benfred/py-spy>.
- [15] Google. 2024. Perfetto: System-wide profiling for Android and Linux. <https://perfetto.dev>.
- [16] Grafana Labs. 2024. Grafana: The open observability platform. <https://grafana.com>.
- [17] Yu Guan, Zhiyu Yin, Haoyu Chen, Sheng Cheng, Chaojie Yang, Kun Qian, Tianyin Xu, Pengcheng Zhang, Yang Zhang, Hanyu Zhao, Yong Li, Dennis Cai, and Ennan Zhai. 2026. EROICA: Online Performance Troubleshooting for Large-scale Model Training. In *23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI 26)*. USENIX Association, Renton, WA, 1113–1130.
- [18] Qinghao Hu, Zhisheng Ye, Zerui Wang, Quoteng Wang, Meng Zhang, Qiaoling Chen, Peng Sun, Dahua Lin, Xiaolin Wang, Yingwei Luo, Yonggang Wen, and Tianwei Zhang. 2024. Characterization of Large Language Model Development in the Datacenter. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*. USENIX Association, Santa Clara, CA, 709–729.
- [19] Songlin Huang and Chenshu Wu. 2025. Neutrino: Fine-grained GPU Kernel Profiling via Programmable Probing. In *19th USENIX Symposium on Operating Systems Design and Implementation (OSDI 25)*. USENIX Association, Boston, MA, 331–344.
- [20] Yanping Huang, Youlong Cheng, Ankur Bapna, Orhan Firat, Dehao Chen, Mia Chen, HyoukJoong Lee, Jiquan Ngiam, Quoc V. Le, Yonghui Wu, and Zhifeng Chen. 2019. GPipe: Efficient Training of Giant Neural Networks Using Pipeline Parallelism. In *Advances in Neural Information Processing Systems 32 (NeurIPS)*. Curran Associates, Inc., Vancouver, BC, Canada, 103–112.
- [21] Ziheng Jiang, Haibin Lin, Yinmin Zhong, Qi Huang, Yangrui Chen, Zhi Zhang, Yanghua Peng, Xiang Li, Cong Xie, Shibiao Nong, Yulu Jia,

- Sun He, Hongmin Chen, Zhihao Bai, Qi Hou, Shipeng Yan, Ding Zhou, Yiyao Sheng, Zhuo Jiang, Haohan Xu, Haoran Wei, Zhang Zhang, Pengfei Nie, Leqi Zou, Sida Zhao, Liang Xiang, Zherui Liu, Zhe Li, Xiaoying Jia, Jianxi Ye, Xin Jin, and Xin Liu. 2024. MegaScale: Scaling Large Language Model Training to More Than 10,000 GPUs. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*. USENIX Association, Santa Clara, CA, 745–760.
- [22] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. 2020. Scaling Laws for Neural Language Models. *arXiv preprint arXiv:2001.08361* (2020).
- [23] Apostolos Kokolis, Michael Kuchnik, John Hoffman, Adithya Kumar, Parth Malani, Faye Ma, Zachary DeVito, Shubho Sengupta, Kalyan Saladi, and Carole-Jean Wu. 2024. Revisiting Reliability in Large-Scale Machine Learning Research Clusters. *arXiv preprint arXiv:2410.21680* (2024).
- [24] Jinkun Lin, Ziheng Jiang, Zuquan Song, Sida Zhao, Menghan Yu, Zhanghan Wang, Chenyuan Wang, Zuocheng Shi, Xiang Shi, Wei Jia, Zherui Liu, Shuguang Wang, Haibin Lin, Xin Liu, Aurojit Panda, and Jinyang Li. 2025. Understanding Stragglers in Large Model Training Using What-if Analysis. In *19th USENIX Symposium on Operating Systems Design and Implementation (OSDI 25)*. USENIX Association, Boston, MA, 483–498.
- [25] Meta Research. 2024. Holistic Trace Analysis: A Library to Analyze PyTorch Traces. <https://github.com/facebookresearch/HolisticTraceAnalysis>.
- [26] Deepak Narayanan, Aaron Harlap, Amar Phanishayee, Vivek Seshadri, Nikhil R. Devanur, Gregory R. Ganger, Phillip B. Gibbons, and Matei Zaharia. 2019. PipeDream: Generalized Pipeline Parallelism for DNN Training. In *Proceedings of the 27th ACM Symposium on Operating Systems Principles (SOSP '19)*. Association for Computing Machinery, New York, NY, USA, 1–15.
- [27] Deepak Narayanan, Mohammad Shoeybi, Jared Casper, Patrick LeGresley, Mostofa Patwary, Vijay Korthikanti, Dmitri Vainbrand, Prithvi Kashinkunti, Julie Bernauer, Bryan Catanzaro, Amar Phanishayee, and Matei Zaharia. 2021. Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC '21)*. Association for Computing Machinery, New York, NY, USA, Article 58, 15 pages.
- [28] NVIDIA. 2024. CUDA C++ Programming Guide: Asynchronous Concurrent Execution and Events. <https://docs.nvidia.com/cuda/cuda-c-programming-guide/>.
- [29] NVIDIA. 2024. CUPTI: CUDA Profiling Tools Interface. <https://developer.nvidia.com/cupti>.
- [30] NVIDIA. 2024. NCCL: NVIDIA Collective Communications Library. <https://developer.nvidia.com/nccl>.
- [31] NVIDIA. 2025. NVIDIA CUTLASS Documentation: CuTe DSL Introduction. https://docs.nvidia.com/cutlass/4.4.1/media/docs/pythonDSL/cute_dsl_general/dsl_introduction.html.
- [32] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. 2019. PyTorch: An Imperative Style, High-Performance Deep Learning Library. In *Advances in Neural Information Processing Systems 32 (NeurIPS)*.
- [33] PyTorch. 2025. Kineto: A CPU+GPU Profiling Library. <https://github.com/pytorch/kineto>.
- [34] Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. 2020. ZeRO: Memory Optimizations Toward Training Trillion Parameter Models. In *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 1–16.
- [35] Teven Le Scao, Angela Fan, Christopher Akiki, Ellie Pavlick, Suzana Ilić, Daniel Hesslow, Roman Castagné, Alexandra Sasha Luccioni, François Yvon, Matthias Gallé, et al. 2022. BLOOM: A 176B-Parameter Open-Access Multilingual Language Model. *arXiv preprint arXiv:2211.05100* (2022).
- [36] David W. Scott. 1992. *Multivariate Density Estimation: Theory, Practice, and Visualization*. John Wiley & Sons.
- [37] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. 2019. Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism. In *arXiv preprint arXiv:1909.08053*.
- [38] Tencent Hunyuan Team. 2024. HunYuan-Large: An Open-Source MoE Model with 52 Billion Activated Parameters by Tencent. *arXiv preprint arXiv:2411.02265* (2024).
- [39] John W. Tukey. 1977. *Exploratory Data Analysis*. Addison-Wesley.
- [40] Cédric Villani. 2009. *Optimal Transport: Old and New*. Springer.
- [41] Borui Wan, Gaohong Liu, Zuquan Song, Jun Wang, Yun Zhang, Guangming Sheng, Shuguang Wang, Houmin Wei, Chenyuan Wang, Weiqiang Lou, Xi Yang, Mofan Zhang, Kaihua Jiang, Cheng Ren, Xiaoyun Zhi, Menghan Yu, Zhe Nan, Zhuolin Zheng, Baoquan Zhong, Qinlong Wang, Huan Yu, Jinxin Chi, Wang Zhang, Yuhao Li, Zixian Du, Sida Zhao, Yongqiang Zhang, Jingzhe Tang, Zherui Liu, Chuan Wu, Yanghua Peng, Haibin Lin, Wencong Xiao, Xin Liu, and Liang Xiang. 2025. Robust LLM Training Infrastructure at ByteDance. In *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles (SOSP '25)*. Association for Computing Machinery, 186–203.
- [42] Tianyuan Wu, Wei Wang, Yinghao Yu, Siran Yang, Wenchao Wu, Qinkai Duan, Guodong Yang, Jiamang Wang, Lin Qu, and Liping Zhang. 2025. GREYHOUND: Hunting Fail-Slows in Hybrid-Parallel Training at Scale. In *2025 USENIX Annual Technical Conference (USENIX ATC 25)*. USENIX Association, Boston, MA, 731–747.
- [43] Zhiyi Yao, Pengbo Hu, Congcong Miao, Xuya Jia, Zuning Liang, Yuedong Xu, Chunzhi He, Hao Lu, Mingzhuo Chen, Xiang Li, Zekun He, Yachen Wang, Xianneng Zou, and Juncheng Jiang. 2025. Holmes: Localizing Irregularities in LLM Training with Mega-scale GPU Clusters. In *22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI 25)*. USENIX Association, Philadelphia, PA, 523–540.
- [44] Yanli Zhao, Andrew Gu, Rohan Varma, Liang Luo, Chien-Chin Huang, Min Xu, Less Wright, Hamid Shojanazeri, Mylène Ott, Sam Shleifer, Alban Desmaison, Can Balioglu, Pritam Damania, Bernard Nguyen, Geeta Chauhan, Yuchen Hao, Ajit Mathews, and Shen Li. 2023. PyTorch FSDP: Experiences on Scaling Fully Sharded Data Parallel. *Proceedings of the VLDB Endowment* 16, 12 (2023), 3848–3860.
- [45] Lianmin Zheng, Zhuohan Li, Hao Zhang, Yonghao Zhuang, Zhifeng Chen, Yanping Huang, Yida Wang, Yuanzhong Xu, Danyang Zhuo, Eric P. Xing, Joseph E. Gonzalez, and Ion Stoica. 2022. Alpa: Automating Inter- and Intra-Operator Parallelism for Distributed Deep Learning. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. USENIX Association, Carlsbad, CA, 559–578.

A CUPTI Engineering Optimizations

Beyond the three-path architectural design described in §4.3, ARGUS introduces three additional engineering optimizations to further reduce overhead and improve stability.

Selective injection targeting goal processes. In real training environments, injection-based profiling faces a practical problem: environment variables and runtime contexts are often inherited by auxiliary processes. For example, compilation workers, build toolchains, launcher processes, and child processes spawned by multiprocessing may formally satisfy the conditions for injection, but they are not the actual target workloads that need to be profiled. Indiscriminately enabling CUPTI tracing on these processes not only introduces additional system overhead but also produces large volumes of noise data unrelated to the training workload. Therefore, ARGUS introduces a selective injection mechanism: the system initializes the tracing runtime only when the current process matches the target training workload, such as possessing a distributed worker identity and matching command-line characteristics, skipping auxiliary processes such as compilation workers and launchers.

Pre-allocated buffer reuse. In CUPTI Activity API-based tracing, buffer management itself can become a significant source of additional overhead. Activity data exhibits pronounced high-frequency and bursty characteristics. If memory is allocated and freed in the callback path each time, it not only introduces additional heap allocation overhead but may also cause allocator lock contention, memory fragmentation, and latency jitter. Therefore, ARGUS employs a pre-allocated buffer reuse strategy: the system allocates a fixed number of fixed-size host-side trace buffers once during initialization and reuses them cyclically: CUPTI writes activity records into an idle buffer from the pool; once full, the buffer is handed to the backend for parsing; after parsing completes, the buffer is returned to the pool for reuse. This approach moves memory allocation out of the high-frequency hot path, leaving only lightweight “take buffer” and “return buffer” operations during online collection.

Bounded resources and backpressure. Another common problem in high-frequency tracing systems is that once the backend processing speed cannot keep up with data production, unprocessed data accumulates in memory, eventually causing profiling itself to evolve into a new performance bottleneck. Therefore, ARGUS adopts a bounded resources and backpressure control strategy: critical resources in the system, including the trace buffer pool and the backend export queue, are all constrained within explicit budgets. When the backend processing speed cannot keep pace with the frontend collection rate, the system does not unconditionally expand capacity or queue without limit; instead, it controls the additional cost of profiling within acceptable bounds by explicitly dropping partial data and continuously monitoring buffer state. In practice, buffer sizes are determined based on

production workload characteristics, and no data dropping has been observed in deployment.

B Diagnosis Algorithm Details (L1/L2)

L1: Sliding-window ratio-gated jitter detection. This algorithm identifies time periods with significant fluctuations in the iteration time series, executing in two phases. In the first phase (*sensitivity gating*), a sliding window of width W slides over the time series point by point; for each window position, the ratio $r = \max/\min$ of the values within the window is computed. When r exceeds threshold θ , that window is marked as a candidate anomalous region, and adjacent or overlapping candidate regions are merged into contiguous anomalous intervals. In the second phase (*effective width measurement*), for each merged anomalous interval, the median of all points outside the interval is computed as the baseline b ; within the interval, the longest contiguous subsegment where all points significantly exceed the baseline is identified, and the width of this longest contiguous subsegment is the effective jitter width. The two-phase design resolves the inherent “smearing” effect of sliding windows: when a narrow spike much smaller than W appears, the first phase inevitably expands the candidate region to at least width W . The second phase then precisely recovers the true anomalous time span through baseline exceedance measurement.

L1: Full-scan change-point detection for regression. This algorithm searches for the most significant single change point in the iteration time series. The algorithm iterates through every valid split point t in the sequence, computing the mean μ_L, μ_R and relative standard deviation $\sigma_L/\mu_L, \sigma_R/\mu_R$ for the two segments before and after the split. A valid change point must simultaneously satisfy: the regression ratio μ_R/μ_L exceeds a minimum threshold (ensuring practically meaningful magnitude), and the relative standard deviation on both sides falls below an upper limit (ensuring both segments are internally stable). The valid split point with the largest regression ratio is selected as the detection result.

L2: CV and z-score computation. Let the average duration of a given event on each rank within parallelism group G during the anomalous time window be $\{\bar{x}_r\}_{r \in G}$. ARGUS computes the group mean and standard deviation:

$$\mu_G = \frac{1}{|G|} \sum_{r \in G} \bar{x}_r, \quad \sigma_G = \sqrt{\frac{1}{|G|-1} \sum_{r \in G} (\bar{x}_r - \mu_G)^2} \quad (5)$$

The coefficient of variation $CV = \sigma_G/\mu_G$ quantifies the degree of intra-group inconsistency, classified into three levels: balanced ($CV < 0.02$), mild imbalance ($0.02 \leq CV < 0.05$), and severe imbalance ($CV \geq 0.05$). Each rank’s z-score $z_r = (\bar{x}_r - \mu_G)/\sigma_G$ identifies stragglers when exceeding the threshold.

Table 5. Experimental model configuration.

Parameter	Value
Transformer layers	36
Hidden size	2048
FFN hidden size	6912
Attention heads / KV groups	32 / 4
Number of experts / Top-K	128 / 8
Expert FFN hidden size	768
Sequence length	4096
Micro-batch / Global-batch	1 / 32
Precision	BF16
Parallelism strategy	TP=1, PP=1, EP=8
Optimizer	Distributed Adam (ZeRO-1)

C Experimental Setup

Experiments are conducted on an 8-GPU node with intra-node GPUs interconnected via NVLink. We train the HunYuan-V3 Preview model [38], which is based on a Mixture-of-Experts (MoE) architecture. The primary configuration parameters are summarized in Table 5. Under

this configuration, each GPU holds $128/8 = 16$ local experts, and GPUs exchange tokens via AllToAll communication. Training is run with performance optimizations including overlap-grad-reduce, overlap-param-gather, and moe-permute-fusion enabled.

D Fault Diagnosis Capability

Table 6 summarizes the fault categories, representative symptoms, and corresponding diagnostic pathways through ARGUS’s progressive framework. The system covers four major categories of fail-slow faults encountered in production: compute hardware degradation (GPU frequency throttling, PCIe bandwidth degradation, ECC error correction), communication degradation (NVLink, inter-node congestion, RDMA link quality), host-side issues (Python GC, data loading, CPU contention), and framework/configuration issues (MoE load imbalance, suboptimal overlap).

Table 6. Summary of ARGUS fault diagnosis capabilities. Each fault category is detected through ARGUS’s progressive diagnostic levels (L1–L3), with L4/L5 available for root-cause confirmation.

Category	Fault Type	Tier	Symptom	ARGUS Localization
Compute hardware degradation	GPU frequency throttling	L2+L3	Compute kernels (GEMM, attention) consistently slower on specific rank	Straggler rank identified via CV; anomalous compute kernels flagged at L3
	PCIe bandwidth degradation	L3	Data-transfer kernels (memcpy H2D/D2H) slower on affected rank	Anomaly detected on memory-copy kernels
	ECC error correction overhead	L1+L3	Intermittent iteration spikes; tail latency on specific kernels	Jitter detected at L1; tail anomaly on specific kernels at L3
Communication degradation	NVLink bandwidth degradation	L2+L3	Intra-node AllReduce/ReduceScatter slower on affected rank	Straggler in EP/DP group; communication kernels flagged at L3
	Inter-node network congestion	L1+L2	Iteration time jitter; communication phase prolonged	Jitter at L1; communication phase identified as bottleneck at L2
	RDMA link quality degradation	L3	AllGather/ReduceScatter kernels show persistent slowdown	Anomaly on specific NCCL kernels on affected stream
Host-side issues	Python GC pause	L1+L5	Iteration time spikes; GPU execution normal but iteration prolonged	Jitter at L1; L2 shows no straggler in compute phases; CPU stack confirms GC
	Data loading stall	L2+L5	GPU idle gap before forward-compute	Semantics shows prolonged gap; CPU stack shows I/O wait
	CPU contention (co-location)	L1+L2	Sustained throughput regression on affected node	Regression at L1; all phases of affected rank slow at L2
Framework/configuration	MoE expert load imbalance	L2	moe_experts duration varies across ranks in EP group	CV analysis on EP group identifies imbalanced ranks
	Suboptimal comm-compute overlap	L4	Communication kernels not overlapped with compute in Perfetto trace	Perfetto timeline reveals serialized execution pattern